Техническое задание v2.0

C++ библиотека smart-video-encoder

Версия: 2.0

Назначение: адаптивное видеокодирование с распределением качества между областями кадра на основе внешних результатов CV.

Изменения относительно v1.0

- Добавлен полноценный Rockchip backend на базе Rockchip MPP/RKMPP.
- GitLab CI/CD становится обязательной системой сборки, тестирования и публикации артефактов.
- Тестирование разделено на быстрые CI-тесты, hardware integration, quality/regression, task-aware CV/ALPR benchmarks и nightly/full benchmark.
- Для каждого типа тестов определены конкретные публичные датасеты и наборы видеопоследовательностей.
- Synthetic sequences привязаны к воспроизводимому генератору.

1. Назначение

smart-video-encoder — C++ библиотека адаптивного видеокодирования, распределяющая качество между областями кадра на основе внешних результатов CV.

```
Video Frame
    +
Detections / Tracks / Masks
    +
Object Priority
        │
        ▼
 smart-video-encoder
        │
        ├── ROI generation
        ├── importance map
        ├── temporal smoothing
        ├── QP / quality allocation
        ├── object-first rate control
        └── metadata embedding
              │
              ▼
         H.264 / H.265
```

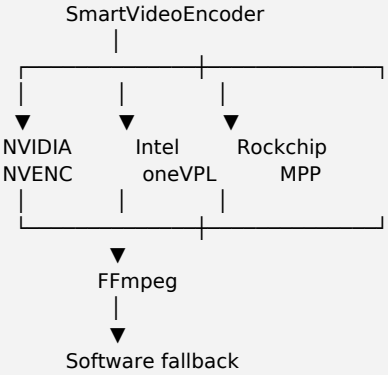
2. Scope

Возможность	Статус
H.264 / AVC	обязательно
H.265 / HEVC	обязательно
NVIDIA NVENC	обязательно
Intel oneVPL	обязательно
Rockchip MPP	обязательно
Software fallback	обязательно
FFmpeg integration	обязательно
ROI	обязательно
Bounding Box	обязательно
Polygon	обязательно
Binary Mask	обязательно

Возможность	Статус
Temporal ROI	обязательно
Object-first mode	обязательно
SEI metadata	обязательно
GitLab CI	обязательно

Архитектурная готовность: H.266 / VVC. Реализация VVC не входит в P0, но добавление backend не должно требовать изменения публичного API.

3. Поддерживаемые backend



4. NVIDIA backend

Технологии: CUDA, NVENC, FFmpeg hwframes interoperability.

- H.264
- H.265
- CUDA input
- zero-copy
- ROI / QP map where supported
- Emphasis map where supported
- capability discovery
- SEI

Конкретный механизм адаптивного качества должен выбираться через capability query, а не жестко предполагаться одинаковым для всех поколений GPU.

5. Intel backend

Технологии: oneVPL, QSV, VAAPI interoperability where required, FFmpeg.

- H.264
- H.265
- mfxExtEncoderROI
- DeltaQP
- runtime capability discovery
- hardware surfaces
- SEI / metadata path

ROI alignment и максимальное число областей должны определяться backend capability/runtime query.

6. Rockchip backend

Добавляется отдельный backend: RockchipMppBackend. Основа — Rockchip Media Process Platform (MPP). MPP предоставляет unified MPI для различных Rockchip SoC и поддерживает аппаратное кодирование H.264/H.265 на соответствующих поколениях encoder hardware; отдельно предусмотрены ROI, CTU-level QP для H.265, rate-control configuration и SEI.

6.1 Обязательные функции

- H.264 / AVC
- H.265 / HEVC
- CPU memory input
- Rockchip DMA-BUF / MPP buffer path where available
- ROI
- QP control
- rate control
- CBR
- VBR
- CQP
- SEI metadata
- capability discovery

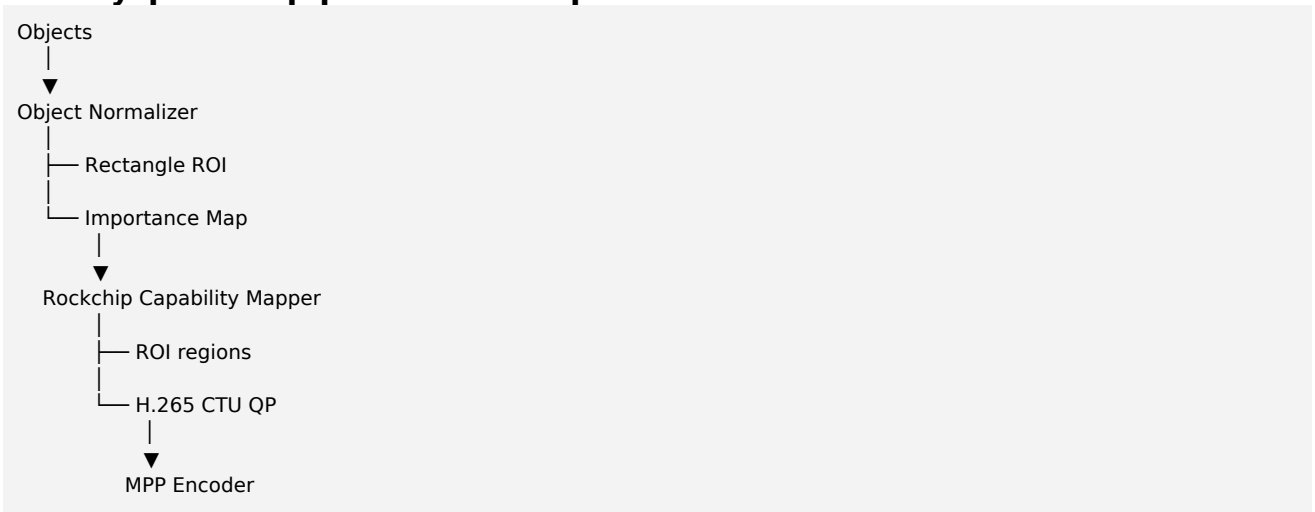
6.2 Rockchip adaptive modes

H.264: ROI rectangles и QP adjustment.

H.265: ROI плюс CTU-level QP where supported.

Архитектура должна позволять backend работать не только с прямоугольными ROI, но и с более детальной block/CTU map. Реальная доступность зависит от конкретного SoC и encoder implementation.

6.3 Внутренний pipeline Rockchip



6.4 Особенность Rockchip

Нельзя предполагать одинаковое число ROI и одинаковый QP API на всех SoC. Поддержка определяется runtime/SoC capability.

```
struct EncoderCapabilities {
    bool h264;
    bool h265;

    bool roi_rectangles;
    bool per_block_qp;
    bool ctu_qp;

    bool runtime_roi_update;
```

```
    bool sei;

    uint32_t max_roi_regions;
};
```

7. Unified Backend API

```
class IEncoderBackend {
public:
    virtual EncoderCapabilities GetCapabilities() = 0;

    virtual Status Configure(
        const EncoderConfig& config) = 0;

    virtual Status SubmitFrame(
        const Frame& frame,
        const BackendFrameControl& control) = 0;

    virtual Result<EncodedPacket> ReceivePacket() = 0;

    virtual Status Flush() = 0;
};
```

Реализации: NvencBackend, OneVplBackend, RockchipMppBackend, FfmpegBackend, SoftwareBackend.

8. Unified Frame API

- AVFrame
- CPU memory
- CUDA memory
- VAAPI surface
- oneVPL surface
- Rockchip MPP buffer / DMA-BUF

```
enum class MemoryType {
    CPU,
    AVFRAME,
    CUDA,
    VAAPI,
    ONEVPL,
    ROCKCHIP_MPP,
    DMABUF
};
```

9. FFmpeg

FFmpeg используется как основной infrastructure layer: AVFrame, AVPacket, demux, decode, mux, format conversion, software fallback и hardware frame interoperability.

FFmpeg также имеет стандартный frame side data для ROI и H.264/H.265 user-data-unregistered SEI. При нескольких ROI порядок и overlap должны учитываться при backend mapping.

10. Object API

```
struct Object {
    uint64_t track_id;

    uint32_t class_id;

    float confidence;

    Geometry geometry;

    float semantic_priority;
    float task_priority;

    bool critical;
```

```
};
```

Геометрия: BoundingBox, Polygon, BinaryMask.

11. Importance model

```
Importance =
SemanticPriority
×
TaskPriority
×
Confidence
×
TemporalStability
×
Optional Motion Weight
```

Результат: I(x, y) ∈ [0, 1].

Объект	Importance
Background	0.05
Road	0.10
Vehicle	0.60
Pedestrian	0.80
Face	0.95
License plate	1.00

12. Nested ROI

Обязательная поддержка.

```
Vehicle
┌──────────────────────────────┐
│              │               │
│     ┌────────┐     │         │
│     │ PLATE  │     │         │
│     └────────┘     │         │
└──────────────────────────────┘
```

Пример: Vehicle QP delta = -4; Plate QP delta = -10. При overlap: highest importance wins, либо configurable composition policy.

13. Temporal ROI

Проблема: detector miss не должен приводить к мгновенному исчезновению ROI.

```
struct TemporalROIConfig {
    uint32_t max_missing_frames;

    float confidence_decay;

    float spatial_smoothing;
    float temporal_smoothing;

    int max_qp_delta_per_frame;

    bool use_track_prediction;
};
```

14. Quality policies

```
enum class QualityPolicy {
    OBJECT_FIRST,
    BALANCED,
    BITRATE_FIRST
};
```

Default: OBJECT_FIRST.

15. Object-first rate control

```
Available bitrate budget
        |
        ▼
Critical objects
        |
        ▼
High-priority objects
        |
        ▼
Normal objects
        |
        ▼
Background
```

```
struct RateControlPolicy {
    uint64_t target_bitrate;

    float max_average_overshoot;
    float max_peak_overshoot;

    uint32_t recovery_window_ms;
};
```

16. Metadata

Для H.264/H.265: User Data Unregistered SEI.

Metadata schema: frame_id, PTS, track_id, class_id, confidence, geometry, priority.

Поддержать Full metadata frame, Delta metadata и Periodic checkpoint.

17. Система тестирования

Тестирование разделяется на шесть уровней: L1 Unit, L2 Component, L3 Software Integration, L4 Hardware Integration, L5 Quality / RD Benchmark, L6 Task-aware CV Benchmark.

18. L1 — Unit tests

- ROI clipping
- ROI alignment
- polygon rasterization
- mask rasterization
- nested ROI
- overlap policy
- importance normalization
- QP mapping
- temporal smoothing
- metadata serialization
- delta metadata
- PTS mapping

Dataset: synthetic generated frames. Все должны выполняться на каждом Merge Request.

19. Synthetic test corpus

Создать отдельный репозиторий или submodule: smart-video-testdata-synthetic. Генерация должна быть deterministic. Использовать FFmpeg и при необходимости OpenCV/custom C++ generator.

- Uniform — constant color; bitrate baseline и encoder stability.

- Linear gradients — banding и low-frequency distortion.
- Checkerboard — high-frequency preservation, blocking, aliasing.
- Zone plate — frequency response, ringing, aliasing, high-frequency loss.
- Multi-frequency sweep — low/medium/high/very high spatial frequency.
- Moving ROI — static background + high-frequency moving object.
- Nested ROI — large object + small critical object.
- ROI flicker — ON/OFF/ON/OFF для temporal persistence.

20. Реальные quality datasets

20.1 BVI-HD

BVI-HD — основной quality dataset. Использовать для PSNR, SSIM, MS-SSIM, VMAF, RD curves, encoder stability, texture preservation и motion sensitivity.

21. BDD100K

Основной dataset для road/surveillance сценариев. Использовать vehicle ROI, pedestrian ROI, traffic scene, day/night, weather, dynamic scene, object-aware compression и detection preservation.

22. MOT17

Основной benchmark для temporal ROI и tracking. Минимальный fast subset: MOT17-02, MOT17-04, MOT17-10, MOT17-13.

Sequence	Назначение
MOT17-02	pedestrian flow
MOT17-04	высокая плотность
MOT17-10	night / moving camera
MOT17-13	traffic/intersection

Метрики: HOTA, IDF1, MOTA, Detection Precision, Detection Recall, mAP where detector evaluation applicable.

23. ALPR dataset — UFPR-ALPR

Использовать как обязательный P0 benchmark для license plate priority. Dataset содержит 150 video sequences и 4,500 annotated frames; использовать vehicle ROI, plate nested ROI, plate detection, OCR accuracy и temporal plate quality.

24. ALPR test protocol

```
Reference
Baseline encoding
Adaptive encoding
```

Параметры: H.264/H.265, low/medium/high bitrate. Затем decoded video передается в Plate Detector и OCR.

- Plate Detection Recall
- Plate Detection Precision
- Character Error Rate
- Plate Error Rate
- Exact Plate Match

Главный KPI: OCR accuracy / bitrate.

25. Dataset matrix

Dataset	Unit	Quality	ROI	Detection	Tracking	ALPR
Synthetic	✓	✓	✓	✓ synthetic	✓ synthetic	✓ synthetic

Dataset	Unit	Quality	ROI	Detection	Tracking	ALPR
BVI-HD		✓				
BDD100K		✓	✓	✓	optional	
MOT17		✓	✓	✓	✓	
UFPR-ALPR	✓ subset	✓	✓	vehicle	track optional	✓

26. Быстрый CI dataset

Создать datasets/ci/ с synthetic (10 sequences), BVI-HD selected short clips, BDD100K selected 10–20 clips, MOT17 4 short sequence excerpts, UFPR-ALPR 10–20 short clips.

Целевой общий размер fast corpus: 5–20 GB либо хранить исходные datasets отдельно, а в CI использовать заранее подготовленные reproducible clips с checksum manifest.

27. Nightly dataset

```
Synthetic FULL
+
BVI-HD FULL
+
BDD100K benchmark subset
+
MOT17 FULL train
+
UFPR-ALPR FULL
```

28. Release benchmark

```
All synthetic
BVI-HD FULL
BDD100K representative benchmark
MOT17 FULL train
UFPR-ALPR FULL
```

Hardware matrix: NVIDIA, Intel, Rockchip.

29. Functional hardware tests

NVIDIA runner

- H.264 encode
- H.265 encode
- ROI
- object-first
- CUDA input
- zero-copy where configured
- SEI
- 24h soak optional nightly

Intel runner

- H.264 oneVPL
- H.265 oneVPL
- ROI
- DeltaQP
- AVFrame path
- hardware surface path

- SEI

Rockchip runner

Основной P0 target: RK3588 / RK3588S. Дополнительные P1: RK3566, RK3568, RV1106/RV1103, RV1126.

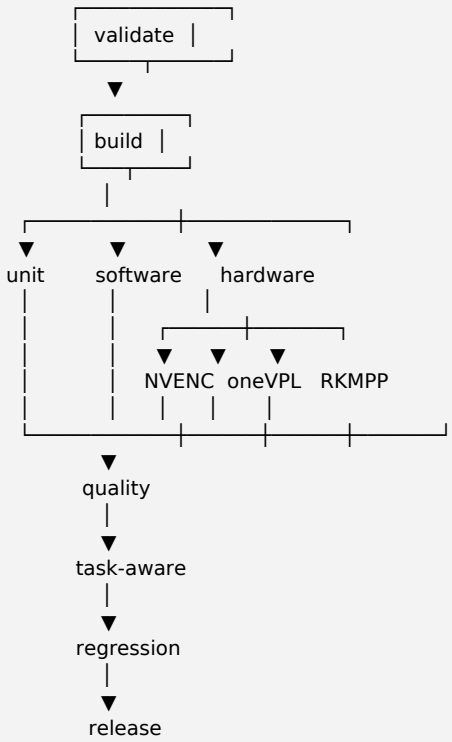
- H.264
- H.265 where hardware supports it
- ROI rectangles
- QP changes
- CTU QP for H.265 where available
- CBR
- VBR
- CQP
- SEI
- DMA-BUF / MPP buffer path where configured

30. GitLab CI/CD

Использовать GitLab CI + GitLab Runner. Основные runner tags: linux-amd64, linux-arm64, nvidia, intel-qsv, rockchip-rk3588.

GitLab jobs должны привязываться к hardware через tags.

31. CI pipeline



32. Pipeline stages

```
stages:
  - validate
  - build
  - unit
  - integration
  - hardware
  - quality
```

```
  - task
  - regression
  - package
```

33. Build matrix

Ubuntu 24.04. Matrix: Software/FFmpeg amd64; NVENC/oneVPL amd64; Rockchip arm64.

```
.build_matrix: &build_matrix
  parallel:
    matrix:
      - BACKEND: [software, ffmpeg]
        ARCH: [amd64]

      - BACKEND: [nvenc, onevpl]
        ARCH: [amd64]

      - BACKEND: [rockchip]
        ARCH: [arm64]
```

34. Hardware runner strategy

```
NVIDIA:
tags:
  - nvidia
  - linux-amd64

Intel:
tags:
  - intel-qsv
  - linux-amd64

Rockchip:
tags:
  - rockchip-rk3588
  - linux-arm64
```

35. Merge Request pipeline

- format
- lint
- static analysis
- unit tests
- software integration
- synthetic fast tests
- small BVI-HD subset
- small MOT17 subset
- small UFPR subset
- hardware smoke tests

Hardware smoke: 10–60 секунд на sequence. Full benchmark на MR не запускать.

36. Nightly pipeline

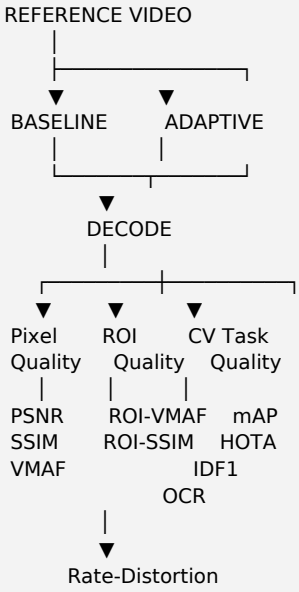
```
all unit
all integration
all hardware
full synthetic
BVI-HD FULL
BDD100K benchmark subset
MOT17 FULL train
UFPR-ALPR FULL
RD curves
```

```
VMAF
CV metrics
performance benchmark
long-running tests
```

37. Weekly pipeline

- 24h stability
- memory leak
- GPU memory leak
- device reset recovery
- long bitrate stability
- large regression corpus

38. Quality test protocol



Для каждого sequence: Original → H.264 baseline/adaptive → H.265 baseline/adaptive. Использовать bitrate points 0.25x, 0.5x, 1x, 2x, 4x относительно nominal bitrate profile для resolution/FPS; фактические points масштабировать по сложности.

39. Quality metrics

Pixel

- PSNR-Y
- PSNR-U
- PSNR-V
- SSIM
- MS-SSIM

Perceptual

- VMAF

Bitstream

- Average bitrate
- Peak bitrate

- Frame size
- P95 frame size
- P99 frame size
- Average overshoot
- Peak overshoot
- Encode FPS
- Latency

40. ROI metrics

- ROI-PSNR
- ROI-SSIM
- ROI-MS-SSIM
- ROI-VMAF
- Background-PSNR
- Background-VMAF
- Global-VMAF

Global VMAF не может быть единственным KPI, поскольку adaptive encoder намеренно может ухудшать background.

41. Quality Separation

```
QSI =
ROI Quality
-
Background Quality
```

Например: ROI-VMAF = 92, Background-VMAF = 70, QSI = 22. QSI не является самостоятельным KPI: высокий QSI при чрезмерно плохом background также считается деградацией.

42. Task-aware benchmark

Главная группа метрик: Bitrate vs Task Quality.

- Bitrate vs mAP
- Bitrate vs HOTA
- Bitrate vs IDF1
- Bitrate vs OCR Exact Match

43. Temporal quality metrics

- ROI-VMAF mean
- ROI-VMAF stddev
- QP stddev
- Importance stddev
- Quality flicker score

```
TemporalQualityInstability =
mean(|Q(t) - Q(t-1)|)
```

Считать отдельно для ROI, Background и Critical objects.

44. ROI boundary test

Для каждого ROI строить зоны: Outside ROI, Boundary outer, Boundary, Boundary inner, ROI interior. Измерять PSNR, SSIM, VMAF и artifact score.

Цель: не допускать резкой видимой границы между ROI и background.

45. Performance metrics

- Input FPS
- Encode FPS
- P50 latency
- P95 latency
- P99 latency
- CPU usage
- CPU RAM
- GPU/accelerator usage
- accelerator memory
- ROI processing time
- importance map generation
- backend mapping
- metadata serialization

Для Rockchip дополнительно: MPP buffer allocation, DMA-BUF path, copy count.

46. Regression framework

Каждый найденный production bug должен превращаться в permanent regression test.

```
tests/regression/

    issue_001_roi_overlap/
    issue_002_pts_reorder/
    issue_003_rockchip_roi_alignment/
    issue_004_nvenc_qp_map/
    issue_005_onevpl_roi_limit/
```

Каждый case содержит input, config, metadata, expected output и metrics thresholds.

47. GitLab artifacts

- encoded video
- JSON metrics
- CSV metrics
- RD curves
- VMAF reports
- CV metrics
- system information
- encoder capabilities
- git commit SHA

```
artifacts/

metrics/
    summary.json
    per_frame.json
    per_track.json

quality/
```

```
    vmaf.json
    psnr.csv
    ssim.csv

task/
    detection.json
    tracking.json
    alpr.json

performance/
    latency.csv
    throughput.csv

environment/
    system.json
    encoder_caps.json
```

48. Acceptance criteria

Build

Проект должен собираться через CMake + CMake Presets + vcpkg manifest mode на Ubuntu 24.04 amd64 и Ubuntu 24.04 arm64.

Backend coverage

Backend	H.264	H.265	ROI
NVENC	✓	✓	capability dependent
oneVPL	✓	✓	✓
Rockchip MPP	✓	✓ where supported	✓
Software	✓	✓	fallback

Functional

- encode/decode succeeds
- correct PTS
- correct frame count
- valid bitstream
- ROI accepted
- ROI affects encoding behavior
- SEI round-trip works

Adaptive quality

На synthetic ROI corpus ROI quality(adaptive) > ROI quality(baseline) при одинаковом nominal bitrate point. Порог фиксируется отдельно для каждого backend после baseline calibration.

Object-first

На UFPR-ALPR OCR Exact Match(adaptive) >= OCR Exact Match(baseline), либо Plate Recall(adaptive) >= baseline при comparable average bitrate либо в пределах заданного overshoot policy.

Tracking

На MOT17 subset: HOTA(adaptive) >= baseline - tolerance; IDF1(adaptive) >= baseline - tolerance. Для critical-object profile качество critical tracks должно быть выше baseline.

Temporal stability

- ROI не должна исчезать мгновенно при detector miss.
- ROI lag при движении объекта должен быть <= configured threshold.

49. Performance overhead

Начальные целевые требования: BBox ROI adaptive overhead <= 10%; Polygon/Mask adaptive overhead <= 20%. Измерять относительно baseline encoder на той же платформе.

50. Rockchip acceptance

На RK3588 baseline: H.264 encode, H.265 encode, ROI enable, multiple ROI, ROI quality difference, SEI, CBR, VBR, CQP, long-run stability. Для H.265 дополнительно CTU QP test, если capability доступна.

51. Long-running acceptance

Nightly/weekly — 24 часа. Проверить memory growth, GPU memory growth, MPP buffer growth, queue growth, latency drift, bitrate drift и metadata desynchronization.

52. Рекомендуемая структура проекта

```
smart-video-encoder/

├── CMakeLists.txt
├── CMakePresets.json
├── vcpkg.json
├── vcpkg-configuration.json
├── .gitlab-ci.yml

├── cmake/

├── include/
│   └── smartvideo/

├── src/
│   ├── core/
│   ├── adaptive/
│   │   ├── object_normalizer.cpp
│   │   ├── importance_map.cpp
│   │   ├── temporal_roi.cpp
│   │   ├── qp_mapper.cpp
│   │   └── bitrate_controller.cpp
│   ├── backend/
│   │   ├── common/
│   │   ├── nvenc/
│   │   ├── onevpl/
│   │   ├── rockchip_mpp/
│   │   ├── ffmpeg/
│   │   └── software/
│   └── metadata/

├── tests/
│   ├── unit/
│   ├── component/
│   ├── integration/
│   ├── hardware/
│   │   ├── nvenc/
│   │   ├── onevpl/
│   │   └── rockchip/
│   ├── quality/
│   ├── task/
│   └── regression/

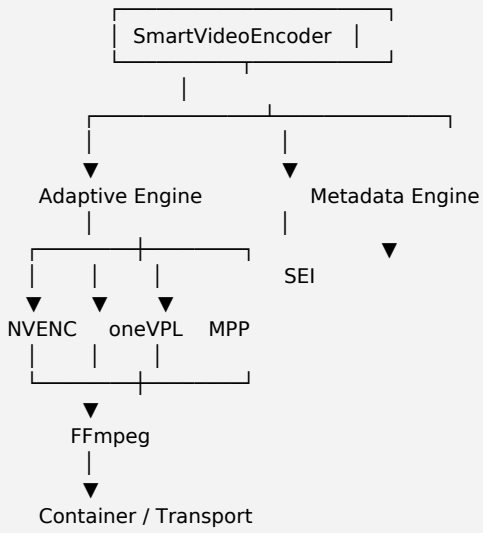
├── benchmarks/
│   ├── synthetic/
│   ├── bvi_hd/
│   ├── bdd100k/
│   ├── mot17/
│   └── ufpr_alpr/

├── testdata/
│   ├── manifests/
│   └── synthetic/
```

```
├── ci/
│   ├── build.yml
│   ├── unit.yml
│   ├── hardware.yml
│   ├── quality.yml
│   └── benchmark.yml

└── docker/
    ├── cpu/
    ├── nvidia/
    ├── intel/
    └── rockchip/
```

53. Итоговая архитектура



54. Итоговые тестовые данные и приоритет

Для P0 использовать Synthetic + BVI-HD + BDD100K + MOT17 + UFPR-ALPR. Набор покрывает классическое качество кодека, пространственную ROI-адаптацию, temporal stability, detection/tracking и сценарий nested vehicle → license plate ROI → OCR quality.

Следующий логичный этап — Epic → Issue → Sub-task backlog для GitLab/OpenProject с порядком реализации core → FFmpeg → NVENC → oneVPL → Rockchip → adaptive ROI → metrics → benchmark, зависимостями и критериями приемки каждой задачи.